Canvas MCP connector — plan

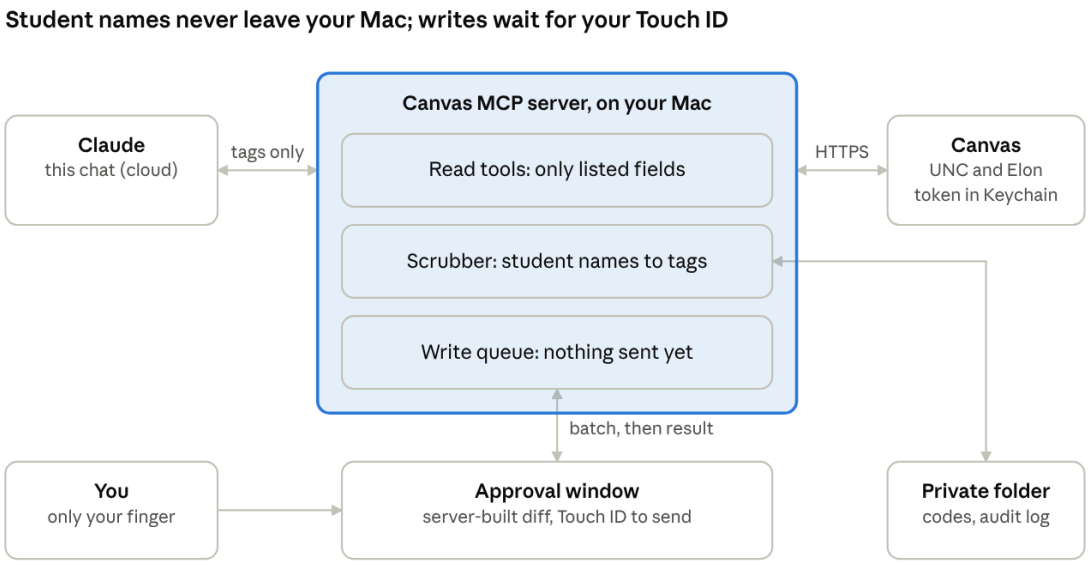
Oct 25, 2026 ·

Bottom line

Recommended: a small custom MCP server on your Mac that only knows a fixed list of Canvas operations, scrubs every response against your rosters before Claude sees it, and holds every write in a queue until you approve it with Touch ID in a window of its own. The gate and the scrubber live in code that is root-owned, so nothing running as you (including Claude's tools on your Mac) can edit them without your admin password.

Phase 1 (read-only, v1.2.1) is built, has been reviewed by Claude Code, Codex and a final check, and passes 156 checks against a fake Canvas; it is waiting for you to install it. Phases 2–4 are not built. Everything you asked for is possible through the Canvas REST API, with two limits: Canvas has no API for the Gradebook's display settings (the grading parts of it are covered instead, see Tool list), and personal Canvas tokens cannot be scoped, so the server's code is the only thing keeping Claude away from grades and submissions.

How it works



Canvas MCP server · reads, scrubber, write queue, approval

Claude talks only to the server, and only in tags. The server talks to Canvas with your token, swaps names for tags on the way in and tags for names on the way out, and sends a write only after the approval window has your Touch ID. The Claude desktop app must be open on your Mac; that is how your existing local Zotero server already reaches these chats.

De-identification

Every student gets the same code the Mail bridge already gives them (form `S-4F7K`), and every identifier is replaced by a typed tag before anything leaves the server: `{{name:S-4F7K}}`, `{{first:S-4F7K}}`, `{{sortable:S-4F7K}}`, `{{email:S-4F7K}}`, `{{login:S-4F7K}}`, `{{sis:S-4F7K}}`, `{{canvas_id:S-4F7K}}`. I use an ASCII hyphen and double braces so tags can't be confused with ordinary text.

Where the roster comes from. At startup and daily, the server pulls each allowlisted course's student enrollments itself and merges them into the bridge's private code table in `~/Library/Application Support/Claude Bridge/`. Canvas gives several identifiers per student (display name, sortable name, short name, login ID, SIS ID, email, Canvas user ID); all of them are matched.

How scrubbing works, in two layers.

1. Field allowlist: each tool returns only fields named in its code. User objects (`last_edited_by`, revision `edited_by`, enrollments) are dropped or replaced by a role word. Unknown fields never pass.
2. Text scrub: every remaining string (page bodies, titles, descriptions, file names, URLs) is searched for every roster identifier, case-insensitively and in slug forms (`jane-doe`, `jane_doe`, `JaneDoe`), because Canvas builds page URLs from titles.

Writing back. Claude writes tags; after your approval the server swaps them for the real values just before the request goes to Canvas. Tags are accepted only in body and description fields. A tag in a title, page URL, file name or module name is refused, since those turn into slugs and links that are harder to scrub on the way back.

What the scrubber can't catch. Nicknames not on the roster, misspellings, names inside images and PDFs, and names of people who aren't enrolled (a former student mentioned on a page). Mitigations: the server never returns file contents, only metadata; you can add aliases to the roster; and an optional weekly self-test (see Build plan) could run the scrubber over all page text and report any capitalised word pair it didn't recognise, to you only, not to Claude.

Staff. Teachers and TAs are not students, so their names pass through (decided).

Approval gate

No write reaches Canvas unless you have touched the Touch ID sensor for that exact batch. Claude has no tool that sends a write; it only has tools that *stage* one.

1. Claude calls a staging tool (for example `stage_page_update`). The server validates it, builds the exact HTTP request it would send, stores it in a queue, and returns a batch ID. Nothing has happened in Canvas.
2. Claude calls `request_approval(batch)`. The server opens its own approval window on your Mac listing every request in the batch: course, item, action, and a before/after diff of each changed field, generated by the server from the stored requests, never from Claude's description of them. Grade-affecting changes and deletions are marked in red.
3. You approve the whole batch, or untick items first, then confirm with Touch ID. Touch ID can't be pressed by software, so neither Claude's computer use nor a script can approve.

4. The server sends only requests whose stored hash matches what you approved, records the result, and Claude reads it with `batch_status`. Unapproved batches expire after 24 hours.

Details that make it hold.

- The approval check runs inside the server process, right before the network call. There is no setting, tool or flag that skips it.
- Approval is single-use and bound to a hash of the method, URL and body. Changing anything after approval voids it.
- If Touch ID isn't available (lid closed, external keyboard without a sensor), Apple's biometrics-only policy simply fails. I'd fall back to your Mac login password, which Claude never sees. Untested on macOS 27.
- If you are away from the Mac (Claude on your phone), writes wait in the queue until you are back. Reads still work while the Mac is awake and the desktop app is open.
- Names: the approval window shows tags as codes by default, with a press-and-hold to show the real names you'd be publishing. That keeps names off the screen in case computer use takes a screenshot.

Tool list

Every feature you listed maps to a documented Canvas REST endpoint; the server exposes about 20 typed tools and no general "call any endpoint" tool. Only allowlisted course IDs are accepted.

Your feature	Canvas endpoint (under `/api/v1/courses/:id`)	Gated?	Notes
View pages	`GET /pages`, `GET /pages/:url`	No	Returns title, URL, `published`, `publish_at`, `front_page`, `editing_roles`, updated date, scrubbed body. Drops `last_edited_by`.
Published / visible to students	same	No	Visible = page `published`, and if it sits in a module, that module published and unlocked (`unlock_at`, prerequisites). The server computes this and says why.
Edit / add pages	`PUT /pages/:url`, `POST /pages`	Yes	Deleting pages, modules and module items is allowed (gated); no other deletes. Revisions read with `summary` only.
View modules	`GET /modules?include[]=items&include[]=content_details`	No	Never passes `student_id`, which would return a student's progress.

Your feature	Canvas endpoint (under `/api/v1/courses/:id`)	Gated?	Notes
Edit / add modules and items	`POST/PUT /modules`, `POST/PUT /modules/:id/items`	Yes	Publish state, unlock date, prerequisites, sequential progress, completion requirements, item order.
View assignments	`GET /assignments`, `GET /assignments/:id`	No	Never requests `submission`, `score_statistics` or `assignment_visibility`; drops `needs_grading_count`. Keeps `unpublishable` so Claude knows when submissions exist.
Edit assignments, all settings	`PUT /assignments/:id`, `POST /assignments`	Yes	Dates, points, grading type, submission types, group, peer review, anonymous and moderated grading, `omit_from_final_grade`, `hide_in_gradebook`. Canvas itself blocks some changes once students have submitted.
Due-date overrides	`/assignments/:id/overrides`	Yes	Section overrides fine; individual-student overrides allowed, shown by code only.
Assignment groups and weights	`GET/POST/PUT /assignment_groups`, course `apply_assignment_group_weights`	Yes	Weights and drop rules change students' computed totals immediately: marked red in approval. No group deletes (Canvas deletes the group's assignments unless told otherwise).
Rubrics (create, edit, attach)	GET/POST/PUT /rubrics, POST /rubric_associations	Yes	Exactly the criteria, ratings and points you give. Reads return structure only; assessments and assessor IDs are never requested. An edit replaces all criteria, so approval shows the whole rubric. 'Use for grading' allowed, marked red in approval.
Discussion topics and reply lists	GET/POST/PUT /discussion_topics, POST /discussion_topics/reorder, GET /discussion_topics/:id/entries	Writes yes	Settings: title, your text, published, delayed post, lock date, threaded, post-first, pinned and pin order, ratings, sort order, sections, groups. Replies: never message text. Ungraded discussions list each reply's author code, date and reply count; graded ones give counts only.

Your feature	Canvas endpoint (under `/api/v1/courses/:id`)	Gated?	Notes
			Links: topic links are documented; per-reply links (?entry_id=) untested. The full-view endpoint is never called, since it returns participants' names.
View Gradebook settings	`GET /settings`, course object, `GET /late_policy`, `GET /grading_standards`	No	Covers grading scheme, weighting on/off, hide totals, late and missing policy. Canvas has no documented API for the Gradebook's own display settings.
View course settings	`GET /settings` and course object	No	Course roster fields (`teachers`, `enrollments`, `total_students`) are never requested.
Upload files	`POST /files`, then upload, then confirm	Yes	Only from one outbox folder on your Mac. Approval window warns if a text file contains a roster name. File listing is metadata only.

Deliberately not built: submissions, grades, rubric assessments, SpeedGrader, Gradebook history and custom columns, posting or hiding grades, quiz results, Inbox, discussions (the text of posts; topics, settings and reply lists are covered above) and announcements, people and enrollments pages, analytics. Assignment `post_manually` (whether an assignment's grades post automatically or manually) and the course's hide-totals setting are reported read-only, so Claude can tell you which assignments hold grades back by policy. Changing the posting policy is out of v1 (untested whether REST can set it). Whether grades are currently hidden is recorded per submission. As the one agreed exception, the server reads each assignment's submissions internally and gives Claude only counts (for example "12 graded, 4 not posted"): no scores, comments, names or codes. Built in Phase 3, about 1 hour.

Notifications

The server gets one more read tool, `attention_report`, which runs fixed checks over the allowed data and returns a short de-identified list. Proposed checks:

- Assignments due in the next 7 days that are unpublished, have no due date, or sit in no module.
- Unpublished pages, files or assignments linked from a published module or page.
- Modules or pages scheduled to open in the next 7 days, so you can check them first.
- Weighting on but group weights not adding to 100%.
- Section due dates that differ from the main due date (often a leftover).
- Assignments with grades entered but not yet posted (counts only), so you remember to post them.

Things that would need data you excluded (ungraded submissions, student messages, discussion activity) are left out. Student mail is already covered by the Mail bridge.

How it runs. Your Mac isn't always on, so nothing can check Canvas while it sleeps. I'd use a scheduled task that needs your computer (weekday mornings), which runs `attention_report` when the Mac is awake and the desktop app is open, and adds Todoist tasks for anything new. Missed runs are simply skipped.

Vulnerabilities

The server can only guarantee its own path. Claude has other routes onto your Mac and into your browser, and those need their own fixes; the ones marked "habit" rely on you rather than on code.

Risk	What could happen	Fix	Enforced by
Claude in Chrome	If you're signed into Canvas in the Chrome profile with the extension, Claude could open the Gradebook or People page and see names and grades. There is no user-level site blocklist (only Team/Enterprise admins get one).	Sign out of Canvas in that profile and use Canvas only in Safari or a second Chrome profile without the extension. Check "Your approved sites" has no Canvas domain.	The profile split is enforced; staying signed out is habit
Computer use	Screenshots of Canvas or the approval window would show names.	Approval window shows codes unless you hold to reveal. Don't use computer use on Canvas.	Partly habit
The token	A personal Canvas token can do anything you can, including grades. Canvas can't scope personal tokens.	Stored in the macOS Keychain, readable only by the server; never in a file, env var or chat. Set an expiry at the end of term.	Keychain
Editing the gate	Claude's shell on your Mac could change server code or config to drop the gate.	Install the server, its course allowlist and the approval helper root-owned (outside any connected folder), so changes need your admin password.	macOS file permissions
Logs and code table	The real-name audit log and code table would leak if a connected folder held them.	Keep them in `~/Library/Application Support/`, never connected.	Habit (don't connect that folder)

Risk	What could happen	Fix	Enforced by
Approval fatigue	Approving big batches without reading.	Batches over 15 items split; deletions and grade changes listed first in red.	Server
Injected text	A page or file name could contain instructions aimed at Claude.	Treated as data; the gate means a fooled Claude still can't write without you.	Server
Institution rules	UNC classes grades and schedules as confidential (Tier 2), and either school may restrict personal tokens or AI tools. Grades never leave the server here, but a token policy could still apply.	Before creating tokens, check each school's token and AI-tool policies (I can find the pages).	You

One more limit: codes are pseudonyms, not anonymity. Anyone who knows the class could match an individual extension to a student. That stays between you and Claude, and you've accepted it for individual overrides.

Alternatives weighed

The server itself

Option	Why not (or why)
Custom minimal server (chosen)	Small enough for you or a reviewer to read; every tool and field is on purpose. Cost: we build and test it.
Fork vishalsachdev/canvas-mcp (Python, MIT, local)	Already pseudonymises students (`Student_xxxxxxxx`) with a local reversal file. But it ships submission, rubric and analytics tools, has no approval step that I could find, and uses its own codes. Removing and auditing would take about as long as building. Worth borrowing ideas from.
Add a Canvas module to the Mail/Calendar Shortcuts bridge	Reuses codes, sensitivity check and dialogs. But the 4-minute run limit, slow folder trigger and click-able dialogs fit Canvas batches and file uploads badly.
Cloud-hosted, like the Zotero connector	Ruled out by your local requirement; the roster would live in the cloud.
Claude in Chrome on the Canvas website	Sees real names and grades on screen. Ruled out.

Option	Why not (or why)
Scoped developer key from your school's Canvas admins	The strongest option: the token itself couldn't touch grades. Needs IT to create a key for you, which many schools refuse for individuals. A later upgrade, not a start.

The approval mechanism

Option	Why not (or why)
Touch ID in a server-owned window (chosen)	Only a finger can approve; the window shows what will actually be sent.
Claude asks you in chat	Depends on Claude's behaviour, which you ruled out.
macOS Allow/Deny dialog, as in the bridge	Computer use could click it. Fine for the bridge's lower-stakes actions, not here.
MCP elicitation (a confirm prompt in the Claude app)	The prompt is drawn by the Claude client, so it's still Claude's side of the fence; untested whether it even reaches this cloud chat through your Mac.
One-time code from your phone typed into chat	The code passes through Claude and isn't tied to a specific payload, so a changed request could reuse it.

Build plan

Phase	Delivers	I do	You do	Done when
0. Prep	Facts the design depends on	Read the bridge's code format; look up UNC and Elon token and AI-tool policies	The checklist in Bottom line (30 min)	Code format confirmed, both Canvas instances allow tokens
1. Read-only server	Pages, modules, assignments, groups, settings, file list; scrubber	Write the server and installer; scrub tests with a fake roster; try to get names out myself	Run the installer (admin password, paste token into its Keychain prompt), restart the Claude app (20 min)	I see your courses as tools here, and a test page containing a test name comes back as a tag
2. Approval gate +	Staged writes, Touch ID window	Build the gate; test that unapproved, altered and	Approve and deny a few test batches	Change made only after Touch

Phase	Delivers	I do	You do	Done when
page/module writes		replayed batches are refused	in the sandbox course (20 min)	ID, visible in Canvas
3. Assignments, groups, overrides, uploads, rubrics, discussions	The rest of your list, plus rubrics (adds about 2-3 hours), unposted-grade counts (about 1 hour) and discussions (about 2-3 hours)	Build; grade-affecting warnings	Test batch in sandbox (20 min)	Same as Phase 2
4. Notifications	`attention_report` + weekday scheduled task	Build; set up the task	Approve the task setup (5 min)	First report lands in Todoist

I can verify most steps myself through your Mac (tool list, test reads, refused writes) and will tell you how; you only need to report the Touch ID results, which I can't see.

Optional extras I left out to keep scope down: the weekly name self-test (about 1 hour), a launchd notifier (1 hour), discussions or announcements read-only (2–3 hours, and they bring student text into play).

Perusall (parked idea, Oct 25)

Idea: a Perusall MCP, possibly folded into this server. Parked in Todoist (Systems parking lot); nothing built. Feasibility as researched from Perusall's public docs, untested:

- **Perusall's API is institution-only.** Its docs say it is available only to institutions with an active Perusall for Institutions subscription, using one institution ID plus one API token that has full access to every endpoint for the whole institution. There is no personal or instructor token, so you can't get one yourself.
- **Even with a token, it's a poor fit for FERPA.** The token can't be limited to your courses, and most endpoints return exactly what this plan keeps out: every user's name and email, per-student scores, annotation text, and per-student analytics. Writes include creating users, enrolling/unenrolling students, setting groups and posting comments on a student's behalf. The only clean reads are course names, the library list, document text (your own uploads, not catalog books) and assignment names/parts/deadlines.
- **What this server already covers.** With the LTI 1.3 Canvas integration, Perusall can create a Canvas assignment link and gradebook column for each Perusall assignment, so those show up in this server's assignment reads (names, due dates, published state, and the Phase 3 posted/unposted counts). Whether Perusall deadlines sync into the Canvas due date is untested; check one course once Phase 1 is installed.
- **Ruled out:** Claude in Chrome or computer use on perusall.com (shows names and annotations), and Perusall gradebook exports (contain names).

Recommendation: no Perusall server. If you want more, ask UNC and Elon IT whether they have Perusall for Institutions and would issue a token; only then would a Perusall module behind this server's scrubber and approval gate be worth planning, with reads limited to assignments and deadlines. Sources: Perusall API reference · Perusall Canvas integration guide

Sources

- Pages API · Modules API · Assignments API · Assignment groups API · Courses API · Late policy API · File uploads · canvasapi course reference
- Manage access tokens permission · Canvas release notes 2024-09-21
- Claude in Chrome permissions · Apple: biometrics policy
- Elon LMS transition · UNC Information Classification Standard · canvas-mcp README